Mini Project Report

on

Study Planner & Task Scheduler using Design & Analysis of Algorithms

Submitted by:

Sanvi Chopra- 1262240712

Nazuk Kabra- 1262240679

SY CSE- Panel A

Course : Project Based Learning-II

Course Code : CSE20090

Under the Internal Guidance of

Dr. Sarika Bobde

Department of Computer Engineering and Technology

MIT World Peace University, Kothrud,

Pune 411 038, Maharashtra - India

2025-2026

DEPARTMENT OF COMPUTER ENGINEERING AND TECHNOLOGY

CERTIFICATE

This is to certify that Sanvi Chopra- 1262240712 & Nazuk Kabra- 12622 of SY BTech (Computer Science & Engineering) have completed their mini project titled "Study Planner & Task Scheduler using Design & Analysis of Algorithms" and have submitted this Project Based Learning -II Report for the academic year 2025-26 (Sem-IV)

[Dr. Sarika Bobde]

Mini Project Guide

Department of CET

MIT World Peace University, Pune

Abstract

This project presents the implementation of a **Study Planner and Task Scheduler** that assists students in efficiently organizing their academic tasks using fundamental concepts of **Design and Analysis of Algorithms (DAA)**. The code focuses on optimizing study time, prioritizing tasks based on deadlines and importance, and generating an effective study schedule.

The project combines multiple algorithmic strategies such as:

- **Greedy Algorithm** - Makes the best immediate (local) choice at each step to reach an overall optimal solution.
- **Dynamic Programming approach (0/1 Knapsack)** - Breaks a problem into overlapping subproblems and stores their results to avoid recomputation.
- **Backtracking** - Tries all possible solutions and abandons a path as soon as it is determined to be invalid.
- **Graph-based approach (Topological Sorting)** - Orders tasks based on dependencies so that prerequisites are completed first.

The input consists of course details including name, required hours, and credits, along with the total available study time and prerequisite relationships between courses. The output provides an optimized study schedule showing selected courses, their execution order based on priorities and prerequisites, and the maximum achievable credits within the given time.

Overall, this project highlights the effectiveness of algorithmic techniques in improving code productivity and decision-making.

Contents

Abstract I

List of Figures & Tables II

Chapter No	Particulars	Page No
1	Introduction	5
2	Problem Statement & Objectives	6
3	Algorithm Design	7-9
4	Complexity Analysis	10
5	Implementation Details and Code	11-20
6	Test Cases and Results	21-25
7	Individual Contributions	26
8	Conclusion and Future Scope	27

Introduction

This project focuses on the application of **Design and Analysis of Algorithms (DAA)** to develop an efficient system for study planning and task scheduling. The aim is to utilize core algorithmic paradigms to optimize resource allocation, prioritize tasks, and generate an effective schedule under given constraints.

The system integrates multiple algorithmic techniques to address different aspects of the problem.

- **Greedy Algorithm:** Selects the maximum number of courses that can be completed within deadlines by prioritizing higher-credit tasks first.
- **Dynamic Programming (Knapsack):** Chooses the best combination of courses that maximizes total credits within the given study time.
- **Backtracking:** Generates all possible study schedules and selects the most optimal one based on constraints.
- **Graph-Based Approach (Topological Sorting):** Orders courses based on prerequisites to ensure correct learning sequence.

Through this project, various DAA concepts such as time complexity, optimization strategies, and problem-solving paradigms are analyzed and compared.

The implementation demonstrates how different algorithmic approaches can be combined to solve a complex scheduling problem efficiently, highlighting their practical relevance and performance trade-offs.

Problem Statement & Objectives

Problem Statement: Design and implement a Study Planner and Task Scheduler that efficiently organizes courses based on their required time, priority (credits), and prerequisite relationships, in order to maximize productivity and ensure optimal utilization of available study time using appropriate algorithmic strategies.

Objectives:

1. **To optimize study time:**
   Efficiently utilize the limited available hours by selecting courses in a way that maximizes productivity and avoids time wastage.
2. **To prioritize tasks:**
   Assign importance to courses based on credits and deadlines so that high-value and urgent tasks are completed first.
3. **To generate an efficient schedule:**
   Create a well-structured study plan that balances workload and ensures maximum course completion within constraints.
4. **To handle prerequisites:**
   Maintain the correct order of learning by ensuring that prerequisite courses are completed before dependent ones.
5. **To apply and compare algorithms:**
   Implement multiple algorithmic strategies and analyze their performance to understand their efficiency in solving the problem.

Algorithm Design (Using Classes and Functions)

Classes Used:

- **Task Class:**
  Stores details of each course including name, required hours, and credits.
- **Planner Class:**
  Manages all operations such as adding tasks, scheduling, optimization, and prerequisite handling using different algorithms.

Functions and Flow of Each Process:

a. **Input Process**

- User enters number of courses
- For each course: Input name, hours, and credits
- Store each course in the tasks array

b. **Display Process**

- Traverse all stored tasks
- Print course name, hours, and credits

c. Job Scheduling (Greedy Algorithm)

- Copy tasks into a temporary array
- Sort tasks in decreasing order of credits
- Find maximum deadline (hours)
- Create slots array (initially empty)
- For each task: Assign it to the latest free slot before its deadline
- Print scheduled courses in order

d. 0/1 Knapsack Problem (Dynamic Programming)

- Store hours as weights and credits as values
- Create DP table dp[n+1][W+1]
- Fill table: If the required hours of the course are less than or equal to the available time, compare the value obtained by adding its credits to the remaining time's result with the value obtained by not selecting it, and take the maximum of the two; otherwise, carry forward the previous value without selecting the course.
- Final value dp[n][W] gives maximum credits
- Backtrack to find selected courses
- Print selected courses and maximum credits

e. Add Prerequisites

- Take input pair (u, v)
- Mark edge u → v in adjacency matrix
- Increase in-degree of v

f. Topological Sorting

- Copy in-degree array
- Add all nodes with indegree 0 to queue
- While queue not empty:
    - Remove node
    - Print course
    - Reduce in-degree of adjacent nodes
    - Add new zero-indegree nodes to queue
- Output valid course order

Complexity Analysis

- **Greedy Scheduling:**
  Time Complexity: **$O(n^2)$** (due to bubble sort and slot allocation)
  Space Complexity: **$O(n)$**
- **Dynamic Programming (Knapsack):**
  Time Complexity: **$O(n \times W)$** (n = number of courses, W = total available time)
  Space Complexity: **$O(n \times W)$**
- **Backtracking:**
  Time Complexity: **$O(n!)$** (explores all possible combinations)
  Space Complexity: **$O(n)$** (recursion stack)
- **Topological Sorting (Graph):**
  Time Complexity: **$O(V^2)$** (adjacency matrix traversal)
  Space Complexity: **$O(V^2)$**
- **Overall System:**
  The complexity depends on the chosen algorithm, with Greedy being faster and Dynamic Programming providing optimal results at higher computational cost.

Implementation Details & Code

Implementation Details:

- **Programming Language:** C++
- **Compiler:** g++ (on iMac)
- **IDE/Editor:** Visual Studio Code
- **Libraries Used:** <iostream>, <string>
- **Execution Type:** Console-based (menu-driven program)

Code:

```
#include <iostream>
#include<string>
using namespace std;

#define MAX 1000

class Task {
public:
    string name;    // Name of the job
    int hours;      // Deadline (time slot limit) → hours required
    int credits;    // Profit (priority of job) → credits

    Task() {}

    Task(string n, int h, int c) { //parameterized constructor
        name = n;
        hours = h;
        credits = c;
    }
};
```

```
class Planner {
private:
    Task tasks[MAX];  // Array to store all jobs
    int n;            // Number of jobs

    //Graph for prerequisites
    int adj[MAX][MAX];
    int indegree[MAX];
public:
    Planner() {
        n = 0;
        for(int i=0;i<MAX;i++){
            indegree[i]=0;
            for(int j=0;j<MAX;j++){
                adj[i][j]=0;
            }
        }
    }

    void addTask(Task t);             // add Jobs
    void displayTasks();              // Display jobs
    void greedy();            // Job Scheduling- Greedy Algorithm
    void knapsack(int W); // 0-1 Knapsack Problem- Dynamic Programming
    void addPrerequisite(int u, int v); //Formation of graph
    void topologicalSort();           //Topological sort of the formed graph
};

void Planner::addTask(Task t) {
    if (n < MAX) {
        tasks[n] = t;
        n++;
    }
```

```
}

void Planner::displayTasks() {
    cout<<"Courses: "<<endl;
    for (int i= 0; i< n; i++) {
        cout << tasks[i].name;
        cout<<" Hours: " <<tasks[i].hours;
        cout<<" Credits: "<< tasks[i].credits;
        cout<<endl;
    }
}
// Greedy Algorithm - Job Scheduling with Deadlines
void Planner::greedy() {
    //Decreasing order of credits using Bubble Sort
    Task temp[MAX];
    for(int i=0;i<n;i++){
        temp[i] = tasks[i];
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (temp[j].credits < temp[j + 1].credits) {
                Task t = temp[j];
                temp[j] = temp[j + 1];
                temp[j + 1] = t;
            }
        }
    }

    int max_hours = 0;
    for (int i = 0; i < n; i++) {
        if (temp[i].hours > max_hours)
            max_hours = temp[i].hours;
    }
```

```
    int slot[MAX];
    for (int i = 0; i < max_hours; i++) {
        slot[i] = -1;
    }

    for (int i = 0; i < n; i++) {
        for (int j = temp[i].hours - 1; j >= 0; j--) {
            if (slot[j] == i) {
                slot[j] = i;
                break;
            }
        }
    }

    for (int j = 0; j < max_hours; j++) {
        if (slot[j] == -1) {
            cout << temp[j].name << " ";
        }
    }

    cout << endl;
}

//Dynamic Programming- 0-1 Knapsack Problem
void Planner::knapsack(int m) {

    int p[MAX], w[MAX];
    int dp[MAX][MAX];
    int x[MAX];

    // Store credits & hours
    for (int i = 1; i <= n; i++) {
```

```
    p[i] = tasks[i-1].credits;
    w[i] = tasks[i-1].hours;
}

// Initialize DP table
for (int i = 0; i <= n; i++) {
    for (int j = 0; j <= m; j++) {
        dp[i][j] = 0;
    }
}

for(int k=0;k<MAX;k++){
    x[k]=0;
}

for(int i=1;i<=n;i++){
    for(int j=0;j<=m;j++){
        if(w[i]<=j){
            if(p[i]+dp[i-1][j-w[i]]>dp[i-1][j]){
                dp[i][j]=p[i]+dp[i-1][j-w[i]];
            }
            else{
                dp[i][j]=dp[i-1][j];
            }
        }
        else{
            dp[i][j]=dp[i-1][j];
        }
    }
}

cout<<endl;
cout<<"DP Table:"<<endl;
```

```
    for(int i=0;i<=n;i++){
        for(int j=0;j<=m;j++){
            cout<<dp[i][j]<<" ";
        }
        cout<<endl;
    }
    cout<<endl;
    cout << "Maximum Credits:" <<dp[n][m]<< endl;

    //Backtracking
    int i=n,j=m;
    while (i>0 && j>=0){
        if (dp[i][j]==dp[i-1][j]) {
            x[i]=0;
            i--;
        }
        else{
            x[i]=1;
            if(j-w[i] >= 0)
                j=j-w[i];
            i--;
        }
    }

    cout <<endl;
    cout<<"Selected Courses:"<<endl;
    for (int k=1;k<=n;k++){
        if(x[k]==1) {
            cout<<tasks[k-1].name<<endl;
        }
    }
}
//Graphs- Topological Sort for pre-requisite sorting
```

```
void Planner::addPrerequisite(int u, int v) {
    adj[u][v]=1;
    indegree[v]++;
}

// Topological Sort
void Planner::topologicalSort() {
    int queue[MAX];
    int front=0,rear=0;
    int indeg[MAX];

    for(int i=0;i<n;i++){
        indeg[i] = indegree[i];
    }
    for(int i=n;i<MAX;i++){
        indeg[i] = 0;
    }
    for(int i=0;i<n;i++){
        if(indeg[i]==0){
            queue[rear++] = i;
        }
    }
    cout << "Course Order:"<<endl;
    cout<<endl;

    while(front<rear){
        int u=queue[front++];
        cout<<tasks[u].name<<"    ";
        for(int v=0;v<n;v++){
            if(adj[u][v]==1){
                indeg[v]--;
                if(indeg[v]==0){
                    queue[rear++]=v;
```

```
            }
        }
    }
}
cout<<endl;
cout<<endl;
}
int main() {
    Planner planner;
    int algo;
    int n;
    cout << "Enter number of courses: ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        string name;
        int credits;
        int hours;

        cout << "Enter details for Course " << i+1 << endl;
        cout << "Name: ";
        cin >> name;
        cout << "Credits: ";
        cin >> credits;
        cout << "Hours Required: ";
        cin >> hours;

        planner.addTask(Task(name, hours, credits));
    }
    cout<<endl;
    while(1) {
    cout<<"Enter Algorithm to Run: "<<endl<<"1) Display"<<endl;
            cout<<"2) Greedy Job Scheduling"<<endl;
```

```
        cout<<"3) Dynamic Programming- Knapsack"<<endl;
        cout<<"4) Add Pre-requisites"<<endl;
        cout<<"5) Topological sort of Courses"<<endl;
        cout<<"6) Exit"<<endl;
    cin>>algo;
    switch(algo) {
      case 1: cout<<"Display All Courses"<<endl;
            planner.displayTasks();
            cout<<endl;
            break;
      case 2: cout<<"Jobs Scheduling using Greedy (Hours): "<<endl;
            planner.greedy();
            cout<<endl;
            break;
      case 3: cout<<"Knapsack to find how many courses can be done in given
Hours using ";
            cout<<"Dynamic Programming"<<endl;
            int capacity;
            cout<<endl;
            cout << "Enter total available hours: ";
            cin >> capacity;
            planner.knapsack(capacity);
            cout<<endl;
            break;
      case 4:
        int edges;
        cout<<endl;
        cout<<"Enter number of prerequisites:";
        cin>>edges;
        cout<<"Enter (u v) meaning u before v:"<<endl;
        for(int i=0;i<edges;i++){
            int u,v;
            cin>>u>>v;
```

```
            planner.addPrerequisite(u,v);
        }
        cout<<"Prerequisites added successfully!"<<endl;
        break;
    case 5:
        cout<<"Topological Sort(Course Order):"<<endl;
        planner.topologicalSort();
        break;
    case 6: return 0;
  }
  }
}
```

Test Cases & Results

Test Case 1:

Number of Courses: 3

Course Details:
Maths Credits: 10 Hours: 2
DSA Credits: 8 Hours: 3
DBMS Credits: 5 Hours: 1

Total Available Hours: 4

Number of Prerequisites: 2
Prerequisites:
0 1
1 2

Result 1:

```
Enter number of courses: 3
Enter details for Course 1
Name: MATHS
Credits: 10
Hours Required: 2
Enter details for Course 2
Name: DSA
Credits: 8
Hours Required: 3
Enter details for Course 3
Name: DBMS
Credits: 5
Hours Required: 1

Enter Algorithm to Run:
1) Display
2) Greedy Job Scheduling
3) Dynamic Programming- Knapsack
4) Add Pre-requisites
5) Topological sort of Courses
6) Exit
1
Display All Courses
Courses:
MATHS Hours: 2 Credits: 10
DSA Hours: 3 Credits: 8
DBMS Hours: 1 Credits: 5
```

```
2
Jobs Scheduling using Greedy (Hours):
DBMS MATHS DSA
```

```
3
Knapsack to find how many courses can be done in given Hours u
sing Dynamic Programming

Enter total available hours: 4

DP Table:
0 0 0 0 0
0 0 10 10 10
0 0 10 10 10
0 5 10 15 15

Maximum Credits:15

Selected Courses:
MATHS
DBMS
```

```
4

Enter number of prerequisites:2
Enter (u v) meaning u before v:
0 1
1 2
Prerequisites added successfully!
```

```
5
Topological Sort(Course Order):
Course Order:

MATHS    DSA    DBMS

Enter Algorithm to Run:
1) Display
2) Greedy Job Scheduling
3) Dynamic Programming- Knapsack
4) Add Pre-requisites
5) Topological sort of Courses
6) Exit
6


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 2:

Number of Courses: 4

Course Details:
Maths Credits: 10 Hours: 2
DSA Credits: 8 Hours: 2
OS Credits: 7 Hours: 1
DBMS Credits: 5 Hours: 1

Total Available Hours: 3

Number of Prerequisites: 2
Prerequisites:
0 1
2 3

Result 2:

```
Enter number of courses: 4
Enter details for Course 1
Name: MATHS
Credits: 10
Hours Required: 2
Enter details for Course 2
Name: DSA
Credits: 8
Hours Required: 2
Enter details for Course 3
Name: OS
Credits: 7
Hours Required: 1
Enter details for Course 4
Name: DBMS
Credits: 5
Hours Required: 1

Enter Algorithm to Run:
1) Display
2) Greedy Job Scheduling
3) Dynamic Programming- Knapsack
4) Add Pre-requisites
5) Topological sort of Courses
6) Exit
1
Display All Courses
Courses:
MATHS Hours: 2 Credits: 10
DSA Hours: 2 Credits: 8
OS Hours: 1 Credits: 7
DBMS Hours: 1 Credits: 5
```

```
2
Jobs Scheduling using Greedy (Hours):
DSA MATHS
```

```
3
Knapsack to find how many courses can be done in given Hours u
sing Dynamic Programming

Enter total available hours: 3

DP Table:
0 0 0 0
0 0 10 10
0 0 10 10
0 7 10 17
0 7 12 17

Maximum Credits:17

Selected Courses:
MATHS
OS
```

```
Enter number of prerequisites:2
Enter (u v) meaning u before v:
0 1
2 3
Prerequisites added successfully!
```

```
5
Topological Sort(Course Order):
Course Order:

MATHS    OS    DSA    DBMS

Enter Algorithm to Run:
1) Display
2) Greedy Job Scheduling
3) Dynamic Programming- Knapsack
4) Add Pre-requisites
5) Topological sort of Courses
6) Exit
6


...Program finished with exit code 0
Press ENTER to exit console.
```

Test Case 3:

Number of Courses: 4

Course Details:
AI Credits: 15 Hours: 3
CN Credits: 6 Hourse: 1
OS Credits: 7 Hours: 2
SE Credits: 5 Hours: 2

Total Available Hours: 4

Number of Prerequisites: 2
Prerequisites:
1 0
2 3

Result 3:

```
Enter number of courses: 4
Enter details for Course 1
Name: AI
Credits: 15
Hours Required: 3
Enter details for Course 2
Name: CN
Credits: 6
Hours Required: 1
Enter details for Course 3
Name: OS
Credits: 7
Hours Required: 2
Enter details for Course 4
Name: SE
Credits: 5
Hours Required: 2

Enter Algorithm to Run:
1) Display
2) Greedy Job Scheduling
3) Dynamic Programming- Knapsack
4) Add Pre-requisites
5) Topological sort of Courses
6) Exit
1
Display All Courses
Courses:
AI Hours: 3 Credits: 15
CN Hours: 1 Credits: 6
OS Hours: 2 Credits: 7
SE Hours: 2 Credits: 5
```

```
2
Jobs Scheduling using Greedy (Hours):
CN OS AI
```

```
Knapsack to find how many courses can be done in given Hours us
ing Dynamic Programming

Enter total available hours: 4

DP Table:
0 0 0 0 0
0 0 0 15 15
0 6 6 15 21
0 6 7 15 21
0 6 7 15 21

Maximum Credits:21

Selected Courses:
AI
CN
```

```
4

Enter number of prerequisites:2
Enter (u v) meaning u before v:
1 0
2 3
Prerequisites added successfully!
```

```
5
Topological Sort(Course Order):
Course Order:

CN    OS    AI    SE
```

Individual Contributions

Sanvi Chopra- 1262240712:

- Designed the overall project structure and problem formulation
- Implemented **Dynamic Programming (Knapsack)** logic for optimal course selection
- Developed **Greedy Scheduling algorithm** and handled debugging
- Worked on testing, result analysis, and report documentation

Nazuk Kabra- 1262240679:

- Implemented **Graph-based approach (Topological Sorting)**, **Prerequisites & Backtracking**
- Developed input handling and menu-driven interface
- Assisted in integrating different modules into a single program
- Helped in testing, validation, and report documentation

Conclusion and Future Scope

The Smart Study Planner and Task Scheduler successfully demonstrates the application of Design and Analysis of Algorithms (DAA) in solving a real-world scheduling problem. By integrating multiple algorithmic techniques such as Greedy, Dynamic Programming, and Graph-based approaches, the system efficiently optimizes study time, prioritizes tasks, and maintains prerequisite order. The project highlights how different algorithms can be used together to achieve better performance and decision-making. Overall, it provides a practical understanding of algorithm design, efficiency, and trade-offs.

Future Scope

- Enhance the system to handle **real-time scheduling** with dynamic changes in tasks and deadlines
- Integrate a **Graphical User Interface (GUI)** or web-based interface for better usability
- Incorporate **AI/ML-based prediction** to automatically assign priorities to tasks
- Extend support for **calendar integration** and reminders
- Optimize algorithms further to handle **large-scale datasets efficiently**